Banker's Algorithm

Banker’s Algorithm for multiple resource types.

Consider a system with 4 processes: P1 through P4 and 3 resource types: A (9 units) B (3 units), C (6 units). Current allocation of resources and maximum requirements of a process for each resource are given below. Determine whether the current state of this system is safe or not.

CA =

	A	B	C
P1	1	0	0
P2	6	1	2
P3	2	1	1
P4	0	0	2

MaxReq =

	A	B	C
P1	3	2	2
P2	6	1	3
P3	3	1	4
P4	4	2	2

The request matrix is denoted with R and array A denotes the available resource counts. We assume that all processes request their maximum required resources all at once.

R =

	A	B	C
P1	2	2	2
P2	0	0	1
P3	1	0	3
P4	4	2	0

A = (0 1 1)

Since R(P2) < A (0 1 1), process P2 is served first.

CA =

	A	B	C
P1	1	0	0
P2	6	1	3
P3	2	1	1
P4	0	0	2

and   A = (0 1 0)

P2 runs to completion and releases all its acquired resources. Hence, A = (6 2 3)

R =

	A	B	C
P1	2	2	2
P2	_	_	_
P3	1	0	3
P4	4	2	0

Since R(P1) < A (6 2 3), process P1 is served. [Note that here you can choose P1 or P3 or P4. In case of multiple candidate rows, you can choose any]

CA =

	A	B	C
P1	3	2	2
P2	0	0	0
P3	2	1	1
P4	0	0	2

and A = (4 0 1)

P1 runs to completion and releases all its acquired resources. Hence, A = (7 2 3)

R =

	A	B	C
P1	_	_	_
P2	_	_	_
P3	1	0	3
P4	4	2	0

Since R(P3) < A (7 2 3), process P3 is served.

CA =

	A	B	C
P1	0	0	0
P2	0	0	0
P3	3	1	4
P4	0	0	2

and A = (6 2 0)

P3 runs to completion and releases all its acquired resources. Hence, A = (9 3 4)

Similarly, since R(P4) < A (9 2 3), process P4 is served.

Hence, we obtain a scheduling order: P2 -> P1 -> P3 -> P4.
Since there exists at least one scheduling order such that every process runs to completion even if all of them request their maximum required resources all at once, the given state is a safe state.

Deadlock detection for single resource

Let us consider the resource graph on slide (slide #17). Showing the execution of the detection algorithm for starting node D. [note: The starting node(s) will be given. In the simulation, you only have to show for the given starting nodes. In actual execution, the deadlock detection algorithm is being executed for every node.]

Starting node D:
Initialization
Initial Node <= D
L = { }

Current Node, CN <= D
L = {D}
Outgoing unmarked edges of D are DS and DT. Let's choose DS.

L = {D, S} and Current Node, CN <= S
Since no outgoing edges for S, and S is not the initial node, return to the previous node and remove S from L.

L = {D}, CN <= D
Outgoing unmarked edges of D: DT. We choose DT.

L = {D, T} and Current Node, CN <= T
Outgoing unmarked edge of T: TE. Let's choose TE.

L = {D, T, E} and Current Node, CN <= E
Outgoing unmarked edge of E: EV. Let's choose EV.

L = {D, T, E, V} and Current Node, CN <= V
Outgoing unmarked edge of V: VG. Let's choose VG.

L = {D, T, E, V, G} and Current Node, CN <= G
Outgoing unmarked edge of G: GU. Let's choose GU.

L = {D, T, E, V, G, U} and Current Node, CN <= U
Outgoing unmarked edge of U: UD. Let's choose UD.

L = {D, T, E, V, G, U, D} and Current Node, CN <= D
Since D appeared twice in L, hence the graph contains a cycle. Therefore, a potential deadlock exists.

Note that deadlock does not exist if starting from each node returns the output "no cycle found".

Starting node R:
Initialization
Initial Node <= R
L = { }
`
Current Node, CN <= R
L = {R}
Outgoing unmarked edge of R: RA. Let's choose RA.

L = {R, A}, CN <= A
Outgoing unmarked edge of A: AS. We choose AS.

L = {R, A, S}, CN <= S
Since no outgoing edges for S, and S is not the initial node, return to the previous node and remove S from L.

L = {R, A}, CN <= A
Since no outgoing edges for A, and A is not the initial node, return to the previous node and remove A from L.

L = {R}, CN <= R
Since no outgoing edges for R, and R is the initial node, stop simulation and return "No cycle Found."